

INTEL AI PC · WEB AI 全栈实战 · 演讲稿

你的 AI PC 不只会跑软件

在浏览器里用 WebGPU + WebNN 跑本地 AI 大模型与视觉应用
90 分钟 · 45 页 · 完整口播稿

演示操作键： ← / → 翻页 · N 讲稿开关 · Esc 关讲稿 · F 全屏

P1

你的 AI PC 不只会跑软件 在浏览器里跑本地 AI 大模型与视觉应用

INTEL AI PC · WEB AI 全栈实战 · ⌚ 0:00 – 0:30

开场白：大家好。先问一个问题 —— 你觉得跑一个 AI 大模型，需要什么？很多人第一反应是：一张几万块的显卡，或者一个云端 API key，按 token 付费。

今天我要颠覆这个认知。我们不租服务器、不调云端 API、不花一分钱 token 费用，就用你面前这台笔记本的浏览器，把 AI 模型跑起来。

而且关键词是「AI PC」—— 这台电脑里其实藏着三个能算 AI 的引擎，今天我们就用浏览器把它们一个个唤醒。

P2

这节课，你会带走什么

关于这 90 分钟 · ⌚ 0:30 – 1:30

这节课不是纯理论。讲完你会带走四样东西。

第一是地图。Web AI 这套技术栈名字很多，初学者容易晕。我会用一张图帮你把它们的关系钉死。

第二是硬件直觉。AI PC 这个词满天飞，但很少有人讲清楚它和普通电脑的区别到底在哪—— 答案是 NPU。

第三，也是最重要的：两个你今天就能在自己浏览器里跑起来的 Demo。一个跑大模型对话，一个跑图像识别。

第四是判断力。技术选型里最值钱的不是“会用”，而是“知道什么时候不该用”。本地推理很酷，但它有边界，我会讲清楚。

P3 先看一个反直觉的画面

LIVE DEMO · 现场演示 · 1:30 – 4:00

【现场操作】打开 `track-a-webgpu-llm/index.html`，点加载模型，输入"用一句话解释什么是 AI PC"，让它流式生成。

边演示边说：注意看，文字是一个一个蹦出来的，这是模型在实时生成。现在我打开任务管理器——看 GPU 占用，拉满了。说明这个计算是真的发生在我这台笔记本的 GPU 上，不是偷偷调了云端。

【杀手锏】我现在把 WiFi 关掉……（关网）……再让它生成一次。看，照样能跑。因为模型已经下载到浏览器缓存里了，全程本地。

这就是今天的主题：让浏览器变成一个 AI 运行时。接下来 80 分钟，我拆给你看这是怎么做到的。

如果现场网络不稳，提前缓存好模型；或者用录屏兜底。

P4 云端 vs 本地：三个维度的取舍

为什么要在本地上跑 · 4:00 – 7:00

为什么要折腾本地推理？四个理由它赢，两个理由它输，看这张表。

成本：云端按 token 收费，一个班 50 个学生天天调，账单很快上来。本地是零边际成本，电费而已。

隐私：这是最硬的理由。医疗、法律、个人笔记这类数据，你愿意传给第三方吗？本地推理数据根本不出这台机器。

延迟：云端每次请求都要网络往返，本地是直接算，省掉这一跳。

离线：飞机上、地铁里、内网环境，云端直接歇菜，本地照跑。

但要诚实——本地也有输的地方：跑不了几百 B 的超大模型，本地内存装不下；知识时效也受限于你下载的那个模型。

所以结论不是"本地取代云端"，而是工程师要有判断力：该本地的本地，该云端的云端。这句话这节课会反复出现。

P5 AI PC = 三个能算 AI 的引擎

AI PC 的硬件底座 · ⌚ 7:00 – 9:00

现在讲硬件。AI PC 和普通电脑差在哪？答案就在这三个方块。

CPU，通用计算，什么都能干，但跑 AI 这种海量并行计算，它慢，因为核心数太少。

GPU，几千个小核同时算，特别适合矩阵乘法，是跑大模型的主力。游戏显卡能跑 AI 就是这个原因。

重点是第三个：NPU，神经网络处理单元。这是这一代 AI PC 的标志。它专门为神经网络的固定计算模式设计，最大的优点是能效——同样跑一个 AI 任务，NPU 的功耗远低于 GPU，风扇都不用转，笔记本续航友好。

记住这个分工：GPU 强在峰值算力，NPU 强在能效。后面你会看到 WebGPU 对应 GPU，WebNN 重点对应 NPU。

P6 从硬件到一行代码：完整技术栈

今天的地图 · 先给全景 · ⌚ 9:00 – 10:00

这是今天最重要的一张图，是地图。我先给你全景，免得后面讲细节时你迷路。

从下往上看：最底下是 AI PC 的硬件，CPU/GPU/NPU。

往上一层，是浏览器标准 WebGPU 和 WebNN，它们是软件摸到硬件的桥梁——浏览器有沙箱，本来碰不到底层硬件，这两个标准就是开的口子。

再往上，是推理引擎层——主流是 ONNX Runtime Web，Google 今年还推了 LiteRT.js。它们把下面两个桥梁统一起来，让你能一键切换用哪个。

最上面，Transformers.js，把一切简化成一行代码。

这张图我会讲三遍：现在记个印象，每讲完一层回来标一次，最后收尾再完整走一遍。你只要记住“从一行代码，一路落到硬件”这个方向感就行。

P7 接下来 80 分钟怎么走

AGENDA · 路线图 · ⌚ 10:00 – 10:30

路线图。我们自底向上走那张图：先讲两个桥梁 WebGPU 和 WebNN，再讲中层引擎 ONNX Runtime Web 和 LiteRT.js，然后是最上层的 Transformers.js。

四块知识讲完，留出 17 分钟动手 —— 你们二选一，要么用 WebGPU 跑大模型对话，要么用 WebNN 跑图像识别。最后总结 + 答疑。

节奏提示：理论部分我会讲得快，重点在让你建立直觉，不抠 API 细节。细节文档里都有，动手环节你自然会查。

P8 WebGPU 浏览器的通用算力

模块一 · 10–25 min · ⌚ 10:30 – 11:00

第一个模块，WebGPU。一句话定位：它是浏览器摸到 GPU 通用算力的那把钥匙。

这一节的目标不是教你写 GPU 代码，而是让你理解：为什么跑大模型需要它，它和老的 WebGL 差在哪。

P9 痛点：浏览器原来摸不到 GPU 算力

问题先行 · ⌚ 11:00 – 13:00

讲新技术，我习惯先讲它解决什么痛点，否则你记不住。

浏览器是个沙箱，为了安全，网页不能随便碰你的硬件 —— 想想看，如果任何网页都能直接操控你的 GPU 和内存，那太危险了。所以历史上，网页想用 GPU 算力，没有正经口子。

那 WebGL 呢？WebGL 确实能用 GPU，但它是为画 3D 图形设计的，比如游戏、地图。你想用它做 AI 计算，得把矩阵数据伪装成图片纹理，把计算伪装成“渲染像素”，绕一大圈，代码难写，性能也打折。

所以 WebGPU 出现之前，浏览器里跑 AI 基本只能靠 CPU，也就是 WASM，慢得跑不动大模型。GPU 明明就躺在你电脑里，网页却够不着。这就是 WebGPU 要解决的核心问题。

P10 现代 GPU 接口，原生支持通用计算

WebGPU 是什么 · 🕒 13:00 – 15:00

那 WebGPU 是什么？它是 W3C 的标准，浏览器里的下一代 GPU 接口，定位就是取代 WebGL。

它底层直接对接操作系统的原生图形 API —— Windows 上是 Direct3D 12，Mac 上是 Metal，Linux/安卓上是 Vulkan。所以它性能接近原生，不是玩具。

最关键的一点：它原生支持 Compute Shader，计算着色器。这个词你不用记，记住它的意义：你终于可以直接写“通用并行计算”，而不用像 WebGL 那样伪装成画图。

对 AI 来说，这就是解放。AI 的核心运算是啥？矩阵乘法、张量运算。这些现在能高效地跑在 GPU 上了。

一句话总结：WebGPU 让网页第一次能把 GPU 当成一台正经的计算器，而不只是画图工具。

P11 WebGPU vs WebGL

对比 · 🕒 15:00 – 16:30

直接对比一下新老两代。

设计目的：WebGL 就是为画图的；WebGPU 一开始就把通用计算当成一等公民。

通用计算：WebGL 要绕路伪装，WebGPU 原生支持。

底层：WebGL 对应老的 OpenGL，WebGPU 对应现代的 Vulkan、Metal、D3D12，这是代差。

跑 AI：WebGL 勉强能跑但很慢，WebGPU 能高效跑，甚至能跑起几亿参数的语言模型。

一个类比帮你记：用 WebGL 做 AI，像逼着一个画家用画笔去算数学题，他能算但很别扭；WebGPU 直接给你一台为并行计算设计的机器。

P12 为什么 GPU 适合 AI：并行的力量

建立直觉 · 不写代码 · ⌚ 16:30 – 19:00

不写代码，给你一个直觉，理解为什么 GPU 适合 AI。

想象搬一仓库的箱子。CPU 的方式是：雇几个大力士，一个接一个搬。每个人很强、很聪明，能处理复杂情况，但人少，量大就慢。

GPU 的方式是：雇几千个小工，每人同时搬一个箱子。每个小工很弱、只会干简单活，但人多，一起上，瞬间搬完。

关键在于任务的性质。如果任务是“同一种简单操作，重复几百万次”，那 GPU 这种打法碾压 CPU。

大模型在算什么？剥开看，本质就是海量的矩阵乘法——把输入和权重矩阵相乘，同一种运算重复几百万次。这正是 GPU 的主场。

所以这句话你要记住：LLM 用 WebGPU，是因为它要做海量并行的矩阵乘，而 GPU 天生为此而生。

P13 WebGPU 现在能用了吗？能。

落地现状 · ⌚ 19:00 – 20:30

你可能会问：这玩意儿现在能用吗？还是只是个未来标准？答案是：能用了，已经落地。

Chrome 和 Edge 从 113 版开始默认开启，现在桌面端基本开箱即用。Safari 新版也支持了，iOS 上能跑。Firefox 在推进。

代码里检测很简单，看 navigator 上有没有 gpu 这个属性就行。

课堂提示一句：今天动手环节的轨道 A，跑大语言模型，靠的就是 WebGPU。你们用最新版 Chrome 或 Edge，基本都能直接跑起来，待会儿就验证。

（这一页节奏可以快，主要是给学生吃定心丸：不是画饼，是真能用。）

P14 WebNN API 直达 CPU / GPU / NPU

模块二 · 25-40 min · ⌚ 20:30 – 21:00

第二个模块，WebNN。如果说 WebGPU 是通用算力的钥匙，那 WebNN 是神经网络专用加速的钥匙，而且是目前 Web 端唯一能摸到 NPU 的标准。

这一节我会讲清楚：它和 WebGPU 到底分工有什么不同，以及为什么 Intel AI PC 的 NPU 这么重要。

P15 痛点：通用算力不够省电

问题先行 · ⌚ 21:00 – 23:00

同样先讲痛点。WebGPU 已经很强了，为什么还要 WebNN？

因为 GPU 跑 AI，算力够，但有代价：功耗高、发热大。你笔记本用 GPU 跑模型，跑一会儿风扇就狂转，电量哗哗往下掉。对移动设备这是大问题。

另一个问题：WebGPU 是通用工具，它不知道你在跑神经网络。怎么把一个神经网络拆成高效的 GPU 计算，得开发者或者库自己操心，很复杂。

那能不能更聪明点？神经网络的计算其实特别有规律——来来去去就是卷积、矩阵乘、激活函数这几样。如果我造一块硬件，专门为这几样运算优化，再提供一个标准，让你能把整个网络直接“交给”这块硬件去跑，那不就又快又省电吗？

这块专用硬件就是 NPU，这个标准就是 WebNN。

WebNN 是什么 · 🕒 23:00 – 25:00

WebNN 是什么？W3C 正在标准化的、神经网络专用的 Web API。

它的思路和 WebGPU 很不一样。WebGPU 是给你算力，你自己写怎么算。WebNN 是：你只要描述一张计算图 —— 我这个网络用了卷积、用了矩阵乘、用了哪些激活 —— 然后把这张图交给操作系统的 ML 加速栈，剩下的事，"怎么在硬件上跑得最优"，由系统和硬件去决定。

它能摸到 CPU、GPU，以及最关键的 NPU。划重点：WebNN 是目前 Web 端唯一能直达 NPU 的标准。WebGPU 摸不到 NPU。

一个类比：WebGPU 像租一间厨房，锅碗瓢盆给你，菜你自己做。WebNN 像去餐厅点一道菜，你只说要什么，后厨怎么高效做出来是它的事。对开发者来说，WebNN 更省心，对硬件来说，它能做更激进的优化。

构图思路 · 看个意思 · 🕒 25:00 – 27:00

给你看一眼 WebNN 的代码长什么样，建立印象就行，不用记。

第一步，拿一个加速上下文，注意这里可以指定 `deviceType` 是 `'npu'`，明确告诉系统我想用 NPU。

第二步，用 builder 一个算子一个算子把网络搭出来：定义输入，接一个卷积 `conv2d`，再接一个激活 `relu`。你看，这就是在"描述一张图"，描述这个网络由哪些运算组成。

第三步，`build`，把这张图编译好，交给硬件执行。

但我要强调：实际开发你几乎永远不会手写这段代码。太底层了。真实场景里，ONNX Runtime Web 或者 Transformers.js 会自动帮你生成这些。我给你看，只是让你理解 WebNN 的工作方式 —— 描述图、交给硬件。这个思路记住就够了。

最关键的一张对比 · ⌚ 27:00 – 29:30

这是整个模块最关键的一张表，请认真看。WebGPU 和 WebNN 不是竞争关系，是分工。

本质：WebGPU 是通用算力，WebNN 是神经网络专用加速。

你提供什么：WebGPU 要你自己写计算逻辑；WebNN 你只描述网络结构，省心。

主要硬件：WebGPU 主打 GPU；WebNN 能摸到 NPU，这是它的独门武器。

强项：WebGPU 灵活、峰值算力高；WebNN 能效高、省电。

典型场景：WebGPU 适合大语言模型这种生成任务，计算密集、追求吞吐；WebNN 适合 CNN 视觉、需要长时间持续推理的场景，比如摄像头实时识别，省电就是命。

这正好对应今天的两条轨道：轨道 A 用 WebGPU 跑 LLM，轨道 B 用 WebNN 跑图像分类。我特意这么设计，让你亲手体会两者擅长的不同场景。

最后一句：未来它们会共存，甚至一个应用里 WebGPU 算一部分、WebNN 算一部分，各取所长。

Intel 主线 · 为什么 NPU 重要 · ⌚ 29:30 – 31:30

专门讲 NPU 为什么重要，这是 AI PC 的核心卖点。

GPU 跑 AI 是一团火 —— 峰值算力高，但功耗大、发热高。适合短时间冲刺的任务，算完就歇。

NPU 跑 AI 是一片叶子 —— 它是为神经网络定制的电路，能效比远高于 GPU。同样一个 AI 任务，NPU 可能只用 GPU 几分之一电，而且风扇都不用转。

什么场景需要这个？想想那些要一直开着的 AI 功能：视频会议的背景虚化、实时字幕、本地语音助手、摄像头的人物追踪。这些是常驻的、持续推理的，如果用 GPU，你的笔记本电池撑不过俩小时。用 NPU，它能安静地在后台跑一整天。

这就是为什么 Intel 把 NPU 做进 AI PC，也是为什么 WebNN 能直达 NPU 这件事，对 Web 开发者价值巨大。

再往下挖一层，讲软硬件协同，这是 Intel 这套体系的精髓。

你的网页调 WebNN API，描述一个神经网络。浏览器的 WebNN 实现接到这个请求，在 Windows + Intel 平台上，它可以对接到 Intel 的加速栈。

往下，是 OpenVINO Runtime —— 这是 Intel 的推理优化引擎，专门负责把一个神经网络在 Intel 硬件上跑到最优。再往下，才是真正的 CPU、GPU、NPU。

精妙的地方在于：你写的还是那份标准的 WebNN 代码，完全不用关心底层。但底层 OpenVINO 会智能地把你的计算调度到最合适的硬件上 —— 这个算子放 NPU，那个放 GPU，开发者无感，但硬件全被用上了。

这就是参考课里讲的“软硬件协同”在 Web 端的体现：上层是开放标准，下层是 Intel 的优化引擎，中间无缝衔接。

实话实说 WebNN 的成熟度，不夸大。

标准状态：还是 W3C 的工作草案，在演进，支持的算子还在持续扩展，不是所有模型都能跑。

浏览器：Edge 和 Chrome 里需要手动开开关，去 `chrome://flags` 把那个 `web-machine-learning` 的 flag 打开。不像 WebGPU 那样默认就开。

最佳体验平台：Intel 的 AI PC，带 NPU，配最新驱动，这是 WebNN 跑得最完整的环境。

好消息是降级很优雅：如果你的环境不支持 WebNN，ONNX Runtime Web 会自动回退到 WASM，也就是用 CPU 跑，应用不会崩，只是慢一点。

所以诚实地说：WebNN 比 WebGPU 更前沿、更新。今天轨道 B 我会准备好 WASM 降级方案，保证就算你的浏览器没开 WebNN，也能把 Demo 跑起来，体验到流程。

P22 推理引擎这一层 ONNX Runtime Web ••• LiteRT.js

模块三 · 40–55 min · ⌚ 35:30 – 36:00

第三个模块，推理引擎这一层。主角是 ONNX Runtime Web，简称 ORT Web；讲完它，我会再给你 Google 的 LiteRT.js 做对照。

前面两个模块是底层的桥梁，比较硬核。从这里开始往上走，越来越接近你平时写的代码。这一层引擎的核心价值就一句话：一个模型，三种后端，换一行配置就能换硬件。

我会先用 ORT Web 把这一层讲透，因为它生态最广、今天动手环节就用它；最后两页换成 LiteRT.js，让你看到这一层不是只有一个选项，而且不同引擎长得都差不多。

P23 ONNX：模型的通用格式

先搞懂 ONNX · ⌚ 36:00 – 38:00

先搞懂 ONNX 是什么，这是 ORT Web 的基础。

问题是这样的：模型有的人用 PyTorch 训练，有的人用 TensorFlow，还有别的框架，各家保存格式都不一样。你训练好一个模型想拿到别处部署，格式不兼容，很麻烦。

ONNX 的方案：定一个开放的中间格式。不管你用什么框架训练，训练完都能导出成一个 .onnx 文件，然后这个文件在任何支持 ONNX 的地方都能加载运行。

最好记的类比：ONNX 就是 AI 模型界的 PDF。你用 Word、用 Pages、用什么都行，导成 PDF，对方不管什么设备都能打开。模型导成 ONNX，也是哪儿都能跑——包括我们今天的主角，浏览器。

它在哪一层 · 🕒 38:00 – 40:00

ORT Web 在我们那张大图里处于承上启下的位置，这页讲清楚它两头连着谁。

上面：是你的代码，或者更常见的，是 Transformers.js。它们说"我要跑这个 ONNX 模型"。

ORT Web 在中间：它负责把模型文件加载进来，选一个后端，然后调度执行。

下面：是三种 Execution Provider，执行后端 —— WASM 用 CPU，WebGPU 用 GPU，WebNN 用 NPU/GPU。这就是前两个模块讲的东西，现在它们以"后端选项"的身份出现了。

所以 ORT Web 干的事很清楚：上面只管说要跑什么模型，下面有三种硬件后端，ORT Web 负责把请求落到你选的那个后端上。它是个调度员、翻译官。

核心取舍 · 🕒 40:00 – 42:30

这三个后端怎么选，是工程上的核心取舍，看这张表。

WASM，跑在 CPU 上。优点是兼容性无敌，任何浏览器、任何设备都能跑。缺点是慢。定位是兜底，或者跑很小的模型。

WebGPU，跑在 GPU 上。并行能力强，吞吐高。缺点是需要 GPU 和 WebGPU 支持。适合大模型，尤其是 LLM。

WebNN，跑在 NPU/GPU/CPU 上。优点是能效高、省电。缺点是比较新，算子还在扩展。适合 CNN 视觉任务和需要常驻的推理。

实战里的选择策略很简单：优先用 WebGPU 或 WebNN，如果环境不支持，自动回退到 WASM 兜底。这样一份代码，在好的设备上跑出最优体验，在差的设备上也永远不崩。这是 ORT Web 帮你省心的地方。

看一段真代码 · ⌚ 42:30 – 45:00

看一段真实代码，体会 ORT Web 多简洁。四步。

第一步，引入 onnxruntime-web 这个库。

第二步，创建推理会话，传入模型文件。注意这一行最关键 —— executionProviders 这个数组，就是在这里选后端！我写的是先试 webgpu，不行就回退 wasm。这个数组就是前面讲的"优先用好的，回退兜底"策略的代码体现。

第三步，准备输入。AI 模型的输入是张量 Tensor，这里是一个 $1 \times 3 \times 224 \times 224$ 的图像张量，就是一张 224×224 的彩色图。

第四步，run，跑起来，拿结果。

最想让你记住的一点：如果我想从 GPU 换到 NPU，我只需要改第二步那一行，把 'webgpu' 换成 'webnn'，其他代码一字不动。这就是"换一行就换硬件"的真正含义。模型不用改、输入输出不用改，只换后端。

同一个 ONNX 模型，改一行 executionProviders，就能在 CPU / GPU / NPU 之间切换。

本模块的核心收获 · ⌚ 45:00 – 46:30

这页是 ORT Web 模块的核心收获，我让它单独占一屏，因为太重要。

同一个 ONNX 模型，改一行 executionProviders 配置，就能在 CPU、GPU、NPU 之间自由切换。

想想这件事的意义：在没有 ORT Web 之前，你想从用 CPU 改成用 GPU，可能要换整套代码、换库、甚至换模型格式。现在，硬件选择变成了一个配置项，一个字符串数组。

这就是抽象的力量。ORT Web 把"用什么硬件"这个原本很重的决定，变轻了，变成了一个可以随时调的开关。待会儿动手环节，你们就会亲手切换这个开关，对比不同后端的速度。

同一层的另一个引擎 · ⌚ 46:30 – 47:30

讲完 ORT Web，我必须再给你一个名字：LiteRT.js。因为如果我只讲 ORT Web，你会以为浏览器里只有一个推理引擎，那不对。

背景是这样：Google 在手机端有一个叫 LiteRT 的运行时，以前叫 TensorFlow Lite，优化了很多年，算子写得非常快。2026 年 Google 把这整个运行时编译进了浏览器，就是 LiteRT.js。它要取代的是早年的 TensorFlow.js —— TF.js 的算子是 JavaScript 写的，性能吃亏。

关键是它的定位：和 ORT Web 在同一层。一模一样的角色 —— 加载模型、选后端、调度执行。唯一的区别是模型格式，它吃 .tflite，ORT Web 吃 .onnx。

你看它的三种后端，是不是特别熟悉？wasm 跑 CPU，底层是 XNNPACK，支持多线程和 SIMD；webgpu 跑 GPU，底层是 Google 的 ML Drift；webnn 跑 NPU，目前还是实验阶段。算子不支持就自动回退 CPU。

这说明什么？说明你今天学的不是某一个库的 API，而是这一层的通用结构。换个引擎，这套心智模型照样用。

怎么选 · ⌚ 47:30 – 49:00

那这两个引擎怎么选？看这张表。

模型格式：一个 .onnx，一个 .tflite。这往往就是决定因素 —— 你手上的模型是什么格式，就用哪个引擎，别为了换引擎去转模型。

后端：一模一样，三种。这一列完全不用记。

上层生态：ORT Web 上面有 Transformers.js，Hugging Face 几万个模型一行代码就能调，这是它最大的优势，也是今天动手环节用它的原因。LiteRT.js 这边是和 TensorFlow.js 互操作，另外有个 LiteRT-LM.js 在做浏览器里的大模型。

从 PyTorch 来：ORT Web 要先导出 ONNX。LiteRT.js 用 litert-torch 一步转换，比老的 PyTorch → ONNX → TensorFlow → TF.js 那条链路短很多。

强项：ORT Web 生态最广，LLM 这条路最成熟。LiteRT.js 在视觉、音频这类模型上延迟更好，而且能直接吃手机端调好的量化模型。

有个坑要提醒：LiteRT.js 的 Tensor 需要你手动 delete，输入输出都要。忘了就漏内存。

最后看这行数字：Google 官方基准说，视觉和音频模型上，LiteRT.js 比其他 Web 运行时最高快 3 倍；用上 WebGPU 或 WebNN，相比纯 CPU 能快 5 到 60 倍。但注意 —— 这是在 M4 MacBook Pro 上测的，换设备、换驱动、机器发热了，结果都不一样。所以官方数字只当参考，真要选型，拿你自己的模型在目标机器上测一遍。这是工程习惯。

对照着看代码 · ⌚ 49:00 – 50:30

这页你不用记 API，只要对照着看结构。

第一步，初始化 wasm 运行时，告诉它 wasm 文件放在哪。这步是 ORT Web 没有的，多一行。

第二步，loadAndCompile 编译模型，accelerator 写 webgpu —— 停一下，这一行就是 ORT Web 里的 executionProviders。同一件事，换了个名字。

第三步，new Tensor 准备输入，还是那个 $1 \times 3 \times 224 \times 224$ 的图像张量，和刚才一模一样。

第四步，run 跑起来。这里有个和 ORT Web 不同的细节：结果留在 GPU 上，你得先 moveTo('wasm') 搬回 CPU，才能转成普通数组读出来。然后，输入输出都要手动 delete 释放。

把两页代码放在一起看，你就明白我为什么要讲这个引擎了：引入库、选后端建会话、喂张量、run —— 四步的形状是固定的。这一层不管叫 ORT Web 还是 LiteRT.js，长得都差不多。

所以今天真正值钱的不是“我会写 `ort.InferenceSession.create`”，而是“我知道中层引擎干什么、后端在哪一行选”。这个认知，明年出个新引擎你照样能上手。

模块四 · 55–70 min · ⌚ 50:30 – 51:00

第四个模块，Transformers.js，我们终于爬到栈的最顶层了。

前面三层 —— WebGPU、WebNN、ORT Web —— 听起来都还有点门槛。

Transformers.js 的使命就是把这一切包起来，让你用一行代码就能跑模型。这是离学生最近、最容易上手的一层。

P32 Hugging Face 官方的 JS 版

Transformers.js 是什么 · ⌚ 51:00 – 53:00

Transformers.js 是什么？Hugging Face 官方出的 JavaScript 版。

先说背景：Hugging Face 是全球最大的开源 AI 模型社区，你想要的模型基本都能在上面找到。它的 Python 库叫 transformers，是 AI 圈的标配工具，几乎人人在用。

Transformers.js 就是这个库的官方 JS 版本。妙就妙在：它的 API 设计得和 Python 版几乎一模一样，但它直接在浏览器里跑，底层用的就是我们刚讲的 ONNX Runtime Web。

看右边两段代码对比，Python 和 JavaScript，几乎长得一样——都是 import pipeline，都是 pipeline('sentiment-analysis')，然后调用。

这意味着什么？一个会用 Python transformers 的算法工程师，几乎零学习成本就能把模型搬到浏览器里。这个设计非常聪明，直接复用了整个社区的肌肉记忆。

P33 pipeline(task, model, options)

核心抽象 · ⌚ 53:00 – 55:30

Transformers.js 的核心就是这一个函数：pipeline，三个参数。

第一个参数，任务类型。你要干嘛？文本生成填 text-generation，图像分类填 image-classification，翻译填 translation。Transformers.js 支持几十种任务。

第二个参数，模型名字。直接填 Hugging Face 网站上那个模型的名字，库会自动帮你从 Hub 下载，并且缓存起来。这里我填的是 SmolLM2，一个很小的语言模型。

第三个参数，选项。注意 device 这一行——又是它！'webgpu' 还是 'wasm'，决定用什么硬件。你看，从 ORT Web 的 executionProviders，到这里的 device，"选硬件"这个动作一路传上来，始终是一行配置。

拿到 generator 之后，直接像调函数一样把输入喂进去，就出结果了。三个参数定义一个 AI 能力，一行调用拿到结果。这就是为什么说 Transformers.js 把整套技术栈收敛成了一个 pipeline。

把四个模块串起来 · ⌚ 55:30 – 57:30

现在把今天讲的四个模块，用一次真实的调用串起来，这是那张大图的第二次出现。

你写的，只有最上面这一行：pipeline，文本生成，某个模型，device 设成 webgpu。就这一行。

接下来全自动：Transformers.js 解析你要做文本生成，去 Hub 把模型下载下来，转成 ONNX 交给 ORT Web。

ORT Web 加载这个 .onnx 模型，看到你 device 写的是 webgpu，就选 WebGPU 这个后端。

WebGPU 把计算桥接到 GPU。如果你写的是 webnn，这里就换成 WebNN，桥接到 NPU。

最后，AI PC 的 GPU 或 NPU 真正执行计算，结果一路返回到你那一行代码。

看到没？你只写了第一行，下面三层全自动。这就是抽象层层叠加的威力——每一层都把下面的复杂度藏起来，最后用户只面对一个 pipeline。但今天你理解了底下每一层在干嘛，这就是这节课和“只会调 API”的区别。

P35 下载一次，缓存到底

模型从哪来 · 到哪去 · ⌚ 57:30 – 59:30

讲一下模型的生命周期，这解释了开场那个"拔网线还能跑"的魔术。

首次运行：Transformers.js 从 Hugging Face Hub 把模型权重下载下来，可能几十 MB 到几百 MB，然后存进浏览器的 Cache Storage —— 这是浏览器专门用来缓存大文件的地方。

再次运行：直接从缓存读，秒开，不再重新下载。所以第一次慢，之后都快。

断网之后：模型已经在本地缓存里了，所以完全离线可用。这就是开场我拔掉 WiFi 还能继续生成的原因 —— 模型早下好了，根本不需要网。

最后强调隐私含义：模型下载到本地之后，你每次推理，输入数据是在本地算的，不再上网。你打的字、你上传的图，永远留在这台机器上。对隐私敏感的应用，这是杀手级特性。回扣开头那张"云端vs本地"的表，这就是"数据不出本地"的技术实现。

P36 Transformers.js 支持的任务（部分）

能做的不止聊天 · ⌚ 59:30 – 61:00

别以为 Web AI 只能做聊天机器人。Transformers.js 支持的任务非常广，看这四类。

文本：生成、情感分析、翻译、摘要、问答、分类，传统 NLP 能干的基本都有。

视觉：图像分类、目标检测、图像分割、背景移除、深度估计。背景移除特别实用，做个网页抠图工具，完全本地，不用上传照片。

音频：语音识别（把语音转文字）、语音合成、音频分类。本地字幕、本地语音助手都能做。

多模态：图文检索、给图片生成描述、视觉问答、文本嵌入（做语义搜索的基础）。

关键是，做这么多事，你用的还是同一套 pipeline()，只是换个任务名、换个模型名。而且全在浏览器里、全本地。这给你们的课后作业留了巨大空间 —— 后面会让你们自己挑一个做。

P37



现在，轮到你了

动手实践 · 70–87 min · ⌚ 61:00 – 61:30

理论讲完了，现在是这节课最重要的部分：你自己动手。

接下来你要在自己的浏览器里，亲手跑通一个本地 AI 应用。不是看我跑，是你自己跑。两条轨道，二选一，根据你的兴趣和设备情况选。

P38

两条轨道，对应两个标准

二选一 · 也可都试 · ⌚ 61:30 – 63:00

两条轨道，对应你今天学的两个标准，我特意这么设计。

轨道 A，用 WebGPU 跑大模型。你会在浏览器里和一个本地的语言模型对话。重点观察首 token 延迟和生成速度，然后切换 WebGPU 和 WASM，亲眼看 GPU 比 CPU 快多少。这体现的是 AI PC 的 GPU 算力。

轨道 B，用 WebNN 跑视觉。你上传一张图，让它做图像分类。观察推理耗时，切换 WebNN 和 WASM 对比。这体现的是 AI PC 的 NPU 能效。

怎么选？如果你对大模型、ChatGPT 这类感兴趣，选 A。如果你对图像识别、计算机视觉感兴趣，或者你的机器是带 NPU 的 Intel AI PC，选 B 能体验到 NPU。时间够的同学两个都试。

两个文件我都准备好了，双击就能打开，不需要装任何环境。

轨道 A · 操作步骤 · ⌚ 63:00 – 68:00

轨道 A 的操作步骤，跟着做。

第一步，双击打开 track-a 那个 html 文件。它直接在浏览器里开，不需要服务器。

第二步，device 默认就是 webgpu，点"加载模型"。第一次会下载模型，几十 MB，等进度条走完。这里可以让学生互相帮忙，网络慢的同学耐心等待。

第三步，在输入框打个问题，点"生成"，你会看到文字一个个蹦出来，流式输出。

第四步，记下两个数字：首 token 延迟，就是你点生成到第一个字出现花了多久；还有 tok/s，每秒生成多少 token。

第五步，关键对比：把 device 切成 wasm，重新加载模型，再生成一次。对比 tok/s。你会明显看到 WebGPU 比 WASM 快很多——这就是 GPU 算力的直观体现。

右边是核心代码，其实就是我们讲过的 pipeline，页面已经封装好了，你不用写代码，专注体验和对比这两个后端的差异。

【巡场】这五分钟我会下去转，帮大家解决加载问题。常见问题：网络慢、或者旧浏览器不支持 WebGPU——后者就先用 wasm 体验流程。

轨道 A · 想一想 · ⌚ 68:00 – 69:30

做完轨道 A，停下来想三件事，这是把体验上升成认知的关键。

第一，WebGPU 比 WASM 快多少？看你记的 tok/s。这个差距，就是把计算从 CPU 挪到 GPU 的纯收益，你亲手量出来的。

第二，打开任务管理器，切到性能页看 GPU。生成的时候 GPU 占用会飙升。这是证据——证明计算真的发生在你这台笔记本的 GPU 上，不是我骗你偷偷调云端。

第三，再玩一次断网那个魔术：关 WiFi，点生成，照跑。这时候你应该已经理解为什么了——模型缓存在浏览器里，全程本地，不联网，零成本。

这三点把"我跑了个 Demo"变成"我理解了本地推理的价值"。

轨道 B · 操作步骤 · ⌚ 69:30 – 74:30

轨道 B 的步骤。

第一步，双击打开 track-b 的 html。

第二步，device 选 webnn，点加载模型。这里要诚实提醒：WebNN 比较新，需要在 chrome://flags 里开启那个 flag。如果你没开或者浏览器不支持，没关系，选 wasm，流程完全一样，只是没用到 NPU 而已。我宁愿你跑通流程，也不要卡在环境上。

第三步，上传一张图片，可以用你自己的照片，或者点"用示例图片"。

第四步，点分类，看 Top-5 结果 —— 模型猜这张图是什么，给出前五个可能和概率。同时看推理耗时。

第五步，如果你成功用上了 webnn，切到 wasm 对比一下耗时，再摸摸机器，体会 NPU 的省电和不发热。

右边核心代码，还是 pipeline，任务换成 image-classification，device 换成 webnn 就直达 NPU。一行的事。

【巡场】这一段我重点帮想用 WebNN 的同学开 flag。开不了的就 wasm 兜底，别纠结。

轨道 B · 想一想 · ⌚ 74:30 – 76:00

轨道 B 做完，也想三件事。

第一，你上传的那张图，传出去了吗？没有。整个分类在你浏览器本地完成，照片从没离开你的电脑。对视觉应用，这个隐私意义特别大——想想人脸、想想医疗影像，你绝不希望它们被上传。

第二，如果你对比了 WebNN 和 WASM 的耗时，体会一下 NPU 在视觉推理上的能效。

第三，也是最重要的想象：现在是你上传一张图跑一次。但如果这是实时摄像头识别呢？每秒要跑 30 次，连续跑几小时。这种常驻、高频的场景下，NPU 的省电优势才真正显出价值——用 GPU 电池半小时就干了，用 NPU 能跑一整天。这就是 AI PC 把 NPU 做进去的意义。

好，动手环节到这。大家回到座位，我们收尾。

总结 · 那张图，第三次 · ⌚ 76:00 – 78:00

那张图，第三次，也是最后一次。但这次不一样——现在你不只是看，你已经亲手跑通了它。

从上到下：你写一行 Transformers.js 的 pipeline。它交给 ONNX Runtime Web，这个引擎根据你的 device 设置，选 WebGPU 或 WebNN。WebGPU 桥接到 GPU，WebNN 桥接到 NPU。最后 AI PC 的硬件真正算出结果。

中间这层换成 LiteRT.js 也是同一张图——换引擎不换结构，这是我特意给你讲那两页对照的原因。

回想刚才：轨道 A，你走的是 GPU 这条线，跑大模型；轨道 B，你走的是 NPU 这条线，跑视觉。两条轨道，刚好把这张图的两条路径都跑了一遍。

所以这张图现在对你不是抽象的框框了，是你亲手验证过的东西。这就是这堂课想给你的：不是“会调一个库”，而是“理解一行代码背后，数据怎么一路落到硬件上”。

最后讲判断力，这是工程师最值钱的部分，比会用任何工具都重要。

什么适合放本地？中小模型，几亿参数级别的，本地能装下、能跑动。隐私敏感的数据——照片、笔记、录音，本地处理最安心。离线或弱网场景。高频常驻的推理，比如摄像头、实时字幕。还有，想省云端成本的高频调用——量大的时候本地的零成本很香。

什么还得用云端？超大模型，几百 B 参数那种，本地内存根本装不下。需要最新知识或实时数据的，本地模型是“冻结”在训练那一刻的。需要跨设备同步的重计算。还有对首次加载体积特别敏感的场景——毕竟本地要先下载几十上百 MB 的模型。

记得开头那句话吗？该本地的本地，该云端的云端。绕了一圈，现在你应该真正理解这句话的分量了。技术选型不是追新，是匹配场景。

收尾 · 87-90 min · ⌚ 81:00 – 90:00

收尾，给你们带得走的东西。

资源：Transformers.js 的官方文档做得非常好，有大量能直接跑的示例，强烈建议课后去翻。ONNX Runtime Web、LiteRT.js 的文档和官方博客、WebNN 规范、WebGPU 兼容性查询、Intel 的 AI PC 和 OpenVINO 开发者资料，都列在这了。

LiteRT.js 那个博客里有几个 Demo 值得一看 —— YOLO 目标检测、单目深度估计做实时点云、图像 4 倍超分，都是浏览器里跑的，看完你会对这一层的能力上限有更具体的感觉。

作业：基于今天的轨道 A 或 B，改造出一个你自己的 Web AI 小应用。给你几个方向：本地翻译器、网页版抠图工具、评论区情感分析、本地语音转文字、图片自动打标签。要求三条：纯前端、可离线、数据不出本地 —— 把今天的核心价值用上。

挑一个你真正想用的做，做出来你会很有成就感，因为它真的在你电脑上跑，不花一分钱，还保护隐私。

好，这就是今天全部内容。从“浏览器只能上网”到“浏览器是个 AI 运行时”，希望你带走的不只是两个 Demo，而是那张图和那句判断 —— 该本地的本地，该云端的云端。

现在 Q&A，有什么问题尽管问。

【常见问题预案】

- 模型多大才能在浏览器跑？答：目前舒服的是 1B 参数以内，量化后；2-3B 也有人跑但加载久。
- 手机能跑吗？答：能，但受内存限制，选更小的模型。
- WebNN 什么时候稳定？答：还在演进，关注 W3C 进展和浏览器更新，Intel AI PC 上体验最好。
- 和 WebLLM、MediaPipe 什么关系？答：都是 Web AI 方案，今天讲的是 HF + ORT 这条最通用的链路。